# 一次AI驱动的淘宝客户端需求开发实战

鸿泥 大淘宝技术 2025年10月22日 03:48 浙江 25人



本文揭示了AI Coding在复杂客户端研发中落地的核心方法论。

## 01 引言

随着AI 能力的逐步强大，由 AI 来完成部分需求的 Coding 工作，已成为一种必然技术趋势（微软很早就号称 30% 代码由AI编写）。但受限于客户端业务层本身的复杂性以及业务开发人员对 AI 的了解和熟悉程度，团队中 AI Coding 的使用率并不高，**对 AI 能做什么，怎么让 AI 做，AI 能不能做对**，也存在一些争议。

秉持着**实践是检验真理的唯一标准原则**，团队内从让组内所有客户端同学开始 AI Coding 的实践，到尝试让 AI 完成实际业务代码开发，再到探索进一步 AI 友好型架构和编码模式做了实践的尝试，希望能带来些许启发。

## 02 先用起来

为更好回答AI怎么写好写业务需求，组内发起了一项AI Coding实践活动，让所有一线开发通过使用 oneDayNative 完成一项日常需求的开发，并反馈和分析所有遇到的问题。

### AI Coding 问题汇总




工具部分的问题，随着Coding Agent 方案的不断升级已经在被逐步解决,不在本文的讨论范围内。

> 例如: 近期很火的 Claude Code 通过多 subAgent 的方案理论上可以解决AI Coding中看不全的问题（2.架构理解问题）。Multi Tool并发调用，则可以解决工具低效的问题（3.1 方法查找低效，找不全）。上下文工智能压缩，则可以部分解决上下文超限问题（3.2 上下文超限）

针对工具外 AI Coding中遇到的诸多问题，究其根源往往是AI对项目了解不足或任务描述不满足简单直接的要求，导致的Coding Agent 无法准确找到当前任务需要的所有信息，或关键词信息本身分散，token大量消耗后后出现了AI 幻觉。这种问题除提升Coding Agent 的任务拆解和架构理解能力外，也可通过在任务输入时prompt任务描述，以及提供足够的上下文来部分解决或降低概率。尝试对收集中的问题，提供了一些解决的建议，截取如下。

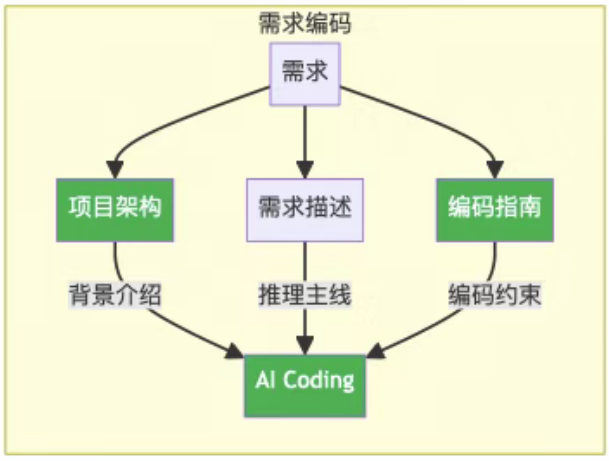
| 优化方向 | 方案 | 解决问题 |
| --- | --- | --- |
| 完善知识库 | 提供《架构描述》 | 2.1 关键词找不到<br>2.3 信息找不全 |
| | 提供《xx模块编码指南》 | 1.3.2 编码不符合业务规范<br>1.2.1 必要协议未履约 |
| | 提供 《swift/java 语言编码》 | 1.3.3 编码不符合通用规范 |
| | 提供 《模块设计规范》<br>（如不能清晰提供，则不建议让AI从事模块设计的工作） | 1.3.1 模块设计不符合规范 |
| | 提供 《外部依赖接入规范》 | 1.1.2 导包报错 |
| | 按需提供必要《UI树/交互流程分析》（帮助AI快速了解相关相关细节） | 1.2.2 业务实现缺失 |
| | 提供 《项目编译指南》 | 1.1 代码编译报错<br>3.4.5 编译指令错误 |
| | | |
| 完善 Prompt | 任务描述： 允许 Claude 说"我不知道" | 1.1.1 捏造代码/返回值 |
| | 任务描述： 使用mermaid流程图 | 1.2.2 业务实现缺失 |
| | 任务描述： 自定义工作流<br>(或 Claude Code 的plan 模式 ) | 1.2.2 业务实现缺失 |
| | checkList: 检测coding 产物符合六大设计原则 | 1.3.3 编码不符合通用规范 |
| | checkList: 检测coding 产物中新增 函数/类/协议 真实存在。 | 1.1.1 捏造代码/返回值 |




> 需求介绍：
> 穿搭动态框架，需要支持多Tab的嵌套方案。主要包括三部分工作:1. UI层支持多Tab嵌套的Template和Container,信息流形式的Component。2. 服务层，新增支持多信息流实例的Service,管理信息流的生命周期和加载，渲染。3. 下拉刷新，点击等交互能力的调试。

这是一个比较经典的客户端需求：涉及UI层，模型层，服务层等全链路的改造。先回答AI 能做什么的问题，就这个需求而言，尝试通过人完成需求的架构设计，由AI来完成各独立业务模块或功能模块的开发（为什么不直接让AI一次性完成整个需求的开发，我们后面讨论）。

考虑到上述AI Coding 问题汇总中出现的各种**AI看不全，写不对**问题，通过在Promt中补充架构描述和编码规范的形式来完善上下文。




## 1. 多信息流实例的Service

在编码前，需充分了解信息流提供了哪些接口，当前模块已有一个InfoFlowService模块负责与信息流一对一的交互，在拓展一个一对多的模块前，需要先充分理解之前的代码设计的。

- 1.1 产出《InfoFlowService模块分析报告.md》

```
## 需求:
分析`InfoFlowService` 模块，梳理该模块的相关依赖和对外能力，并产出一份文档，涉及流
## 要求:
1. 文档中有 所有依赖了InfoFlowService或使用了它能力的模块，用Mermaid产出
2. 文档中有 表述InfoFlowService 在当前架构中的功能介绍。
```

- 1.2 基于通用架构文档 + 指南文档 生成MultiInfoFlowService

```
## 需求:
参考`InfoFlowService` 模块实现，以及本地的《InfoFlowService模块分析报告.md》。
## 要求:
1. 编码前先阅读《模块需知》和所有提到的相关文档。
2. `MultiInfoFlowService`要 满足TurboService的微服务协议。
2. 不可捏造不存在的函数名和类名
3. `MultiInfoFlowService` 中不可依赖`pageDataService`
## 需实现的协议
protocol MultiInfoFlowServiceProtocol : AnyObject {
...
}
```

应用率: 100%

## 2. 新增多 Tab 的Template, Container 和 Component 容器

考虑到 Template ，Conatiner，Component 属于TurboDressing 框架的特殊关键词，在Coding前，需明确相关概念，以及已有UI树的结构，因此在实际编码前也需要先有一份当前库的UI 树结构文档。

- 2.1 产出《TurboDressingFramework UI树结构文档.md》

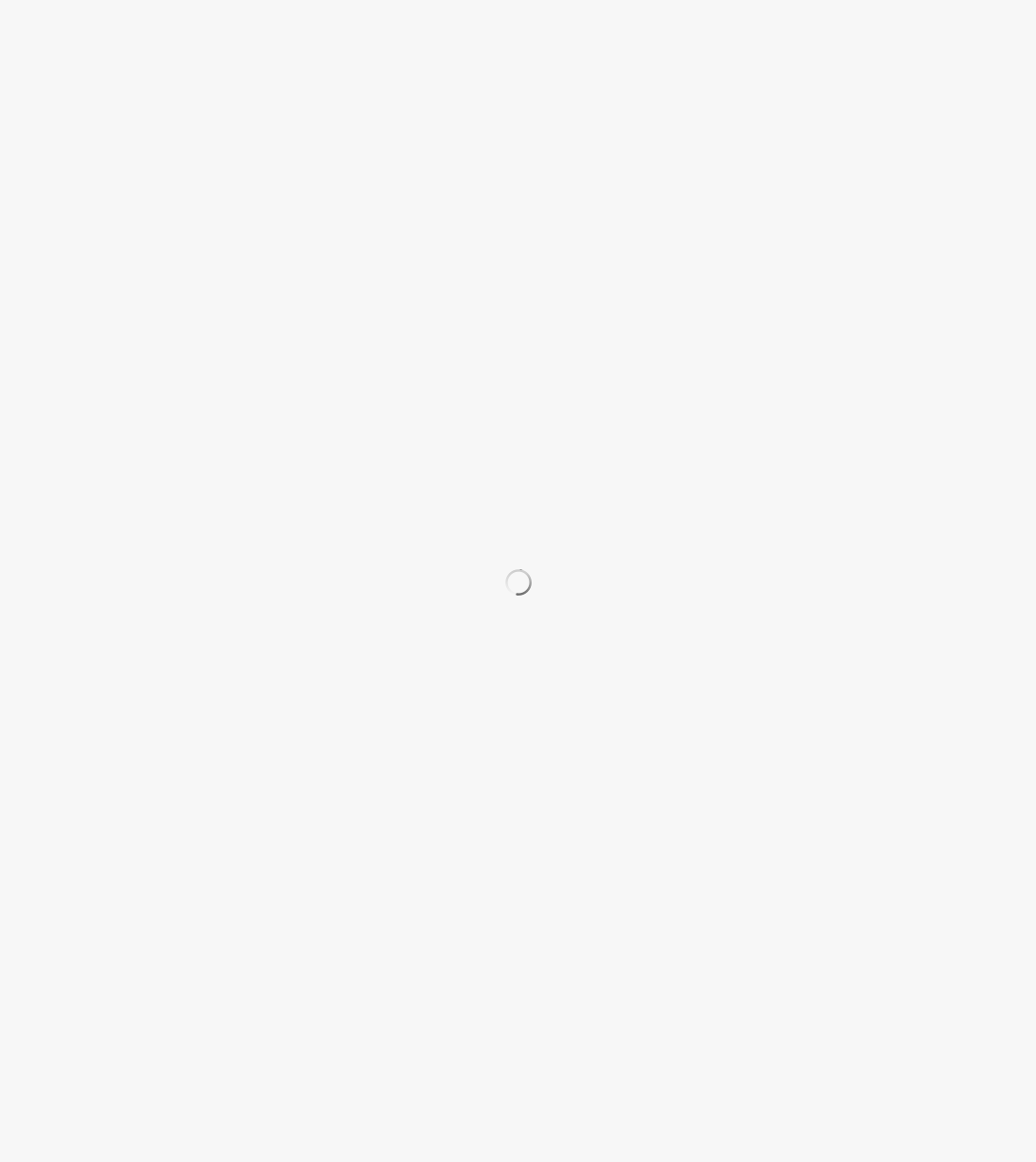
```
## 需求:
理解当前工程的结构，产出当前穿搭业务，从VC到每个子节点的UI树结构的md文档。
## 要求:
1. 注意区分 V1 和 V2 两个版本的UI树都要产出。
2. UI分为 Template/Container/Component。
3. 每层有哪些实例，产出每个实例的类图。
4. 完整阅读完所有相关文档后再开始产出
```

- 2.2 开始多tab的Template和 Container编码

```
##需求:
1. 充分理解当前工程，参考`InfoFlowListTemplate`实现一个`MultiTabTemplate`。
## 要求:
1. 有`header` 和 `tabs`两个Container
2. `MultiTabTemplate` 可上下滚动，基于UITableView实现。
3. `MultiTabTemplate` 只在V2版本中展示。
4. `header` 使用`FeedStackContainer` 承接,内部的`Component`是上下布局的。
5. `tabs` 自定义一个`HorizontalPageContainer`承接
6. 新增的`HorizontalPageContainer`size由`MultiTabTemplate`指定，指定为VC的
7. `HorizontalPageContainer`可左右分页滑动，每一个`Component`占据一个分页。
8. 完整阅读完所有`相关文档`和`指南文档`，编码时严格按照`指南文档`
```

采纳率: 95%

存在问题: Container 触发 Component 生命周期时有bug，会多次触发，人工提醒后修复。



- 2.3 新增InfoFlowComponent用以处理多Tab场景的信息流展示

```
## 需求:
1. 充分理解当前工程, 参考`InfoFlowListTemplate`实现一个`InfoFlowComponent`。
## 要求:
1. `InfoFlowComponent` 会被使用到`HorizontalPageContainer`中
2. `InfoFlowComponent` 渲染通过 `MultiInfoFlowService`实现。
3. 完整阅读完所有`相关文档`和`指南文档`, 编码时严格按照`指南文档`
```

## 3. 拓展数据绑定协议，支持isRepeat字段

- 3.1 先产出一份旧数据绑定协议的文档

```
## 需求:
充分理解当前工程，了解PageModelV2 模型是如何被填充的，并产出概述文档`PageModelV2数
## 要求:
1. 在阅读完所有`相关文档`中提供的文档后，开始推理。
2. 文档中应包含已支持的数据绑定表达式，可参考`BindedDataParser`
```

- 3.2 开始isRepeat协议的编码

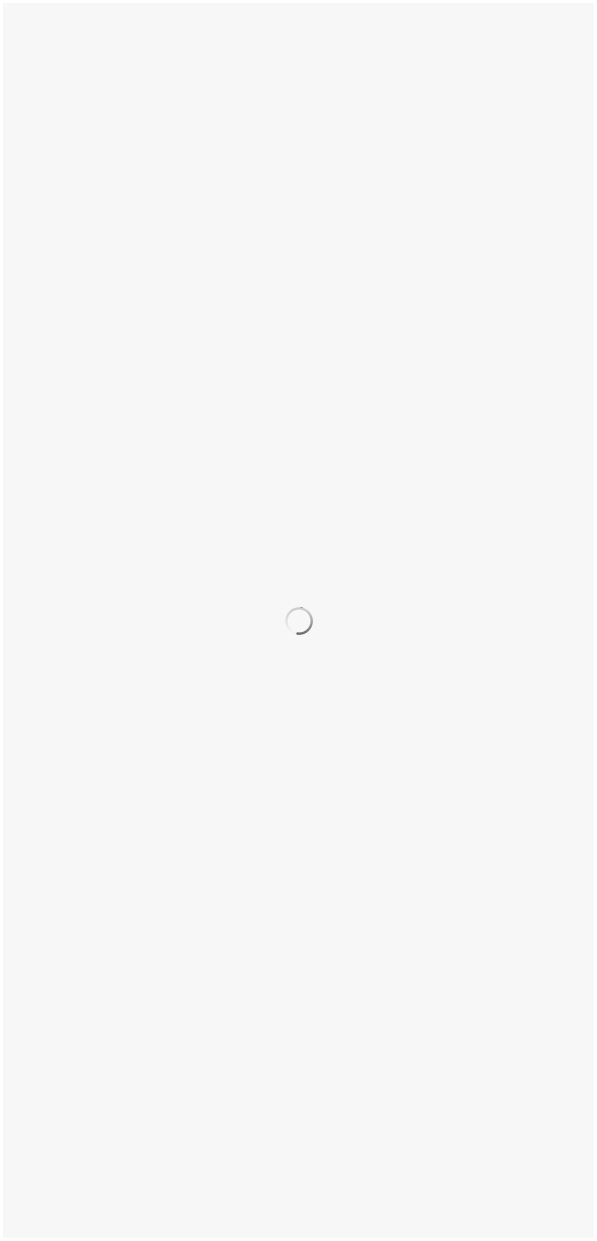
```
## 需求:
1. 充分理解当前工程，拓展PageModelV2的数据绑定协议，在layout层填充CardModel时，
## 数据实例
{"subTemplates":[{"layout":{"tabs":[{"isRepeat":"true","type":"infoflow

## 要求:
1. 阅读完所有相关文档后再开始编码
2. 阅读完本地的`PageModelV2数据绑定规则.md` 后开始编码
```

产物: Commit 记录

采纳率: 95%

存在问题: 代码重复率高，可读性差，提醒后一次修复。



## 4. 效率对比

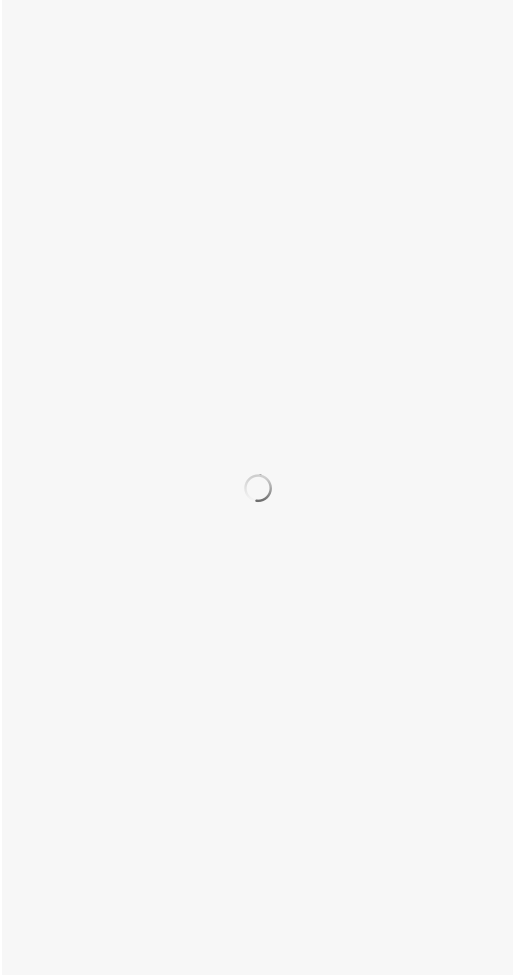
以上AI Coding过程加上人工Check(编译，单侧)总共用时1人日。原需求排期中该部分功能为6人日。

考虑到AI完成的功能都只是单测通过，还需要联合调试，针对调试部分做个33%的预留。

相比人工编码4人日 / AI一人日。**该需求而言，Coding部分效率提升了300%** 。(prompt中引用到的所有文档，均由AI生成)

思考

## 1.什么样的需求交给AI



> 常规的需求研发流程如上图

关于业务需求开发什么阶段让AI接入，参考上述流程图，从上至下抽象度降低，沟通成本减少，确定性增加。在更早的阶段让AI介入就面临更多不确定性风险，需要给到AI的上下文也就越多，相应的AI 产生幻觉和遗忘的概率也就越大。

1. 依赖明确，架构文档完备的模块，可以考虑在架构设计阶段让AI参与，但考虑到AI 的不确定性，AI产物也需要人工不断Check，就实际提效程度而定。
2. 在代码设计和编码阶段，所有的依赖信息已明确，通过合适prompt描述和上下文设计已可以控制AI产出确定和正确的产物了。

## 2. 交给AI的需求如何拆分多子需求

拿到一个业务需求着手AI编码时候，经常面临如何切分需求的问题。具体需求切分的粒度，并没有标准答案。

拿以上实践而言:新增多Tab 的Template和 Container容器和 新增 InfoFlowComponent 用以处理多Tab场景的信息流展示同属 UI逻辑，看起来可以合并为一次AI任务，但我作为两次AI Coding任务出于以下原因考虑。

1. 减少AI Coding 后人工check的工作量，更细的粒度意味产物的高聚合同时低耦合，更容易检测。
2. AI沟通成本高，产物耦合。就这一需求而言我希望Template和Container是业务无关的，Component是耦合信息流业务的。在一次需求中我需要通过种种约束告知AI来处理这一相对主观的·模糊边界。拆分成两个编码任务后，很自然就解耦了。
3. 一次上下文过长，容易让AI产生遗忘，或者产生幻觉，小需求更可控。

就结果而言，好的AI Coding需求应该是可读性，可控性和正确性。拆解成各个高聚合的小Coding任务，易于写任务Prompt的同时，也易于人工对产物的CR。

## 3. 如何写任务的Prompt

Prompt 的质量对AI Coding产物的质量的影响不言而喻。以往实践中发现同一模型基座(Claude 3.7)，一句话需求prompt和上下文清晰约束明确的Prompt就Coding产物的正确性20%～90%之间波动。

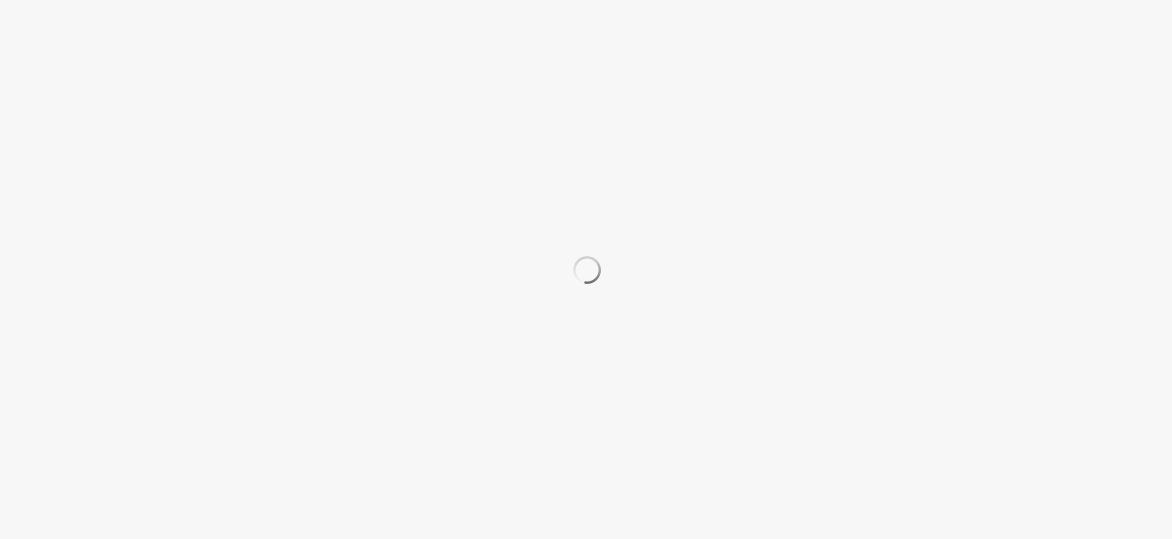
以上几点：

- 1,5,6,7 可以考虑通过 Coding IDE 的实现。
- 3，9 可以通过优化知识库来实现。
- 对人要求比较高的主要是2，4,8。
- 2 :提供清晰和明确的prompt描述，在提供给AI Coding任务尽可能是你能预期输出，给出明确构建描述的任务，当你的Prompt不能指向一个明确的Coding 产物时，可能带来的就是无尽的再次沟通成本，带来的提效时长，在一次次反复补充信息中，逐渐消耗成0甚至不如我自己干了。
- 4 和 8 在详细介绍中都提到了思维链(CoT)，当使用Claude Code时候可以直接开启plan模式。

## 4.知识库如何构建

项目件差异比较大，总体延续 架构描述+编码约束 的思路来对知识库分类。上述业务实践中使用的是 OneDayNative 插件，因此使用的是语雀文档作为知识库，在prompt中手动关联。近期蹭上集团Cursor后，使用Cursor/Generate Cursor Rules 功能也可以一键把原文档变为项目的 project rules。

- Rules



## 5.业务开发范式

- 以穿搭业务为例

穿搭是一套基于TurboFlow 微服务框架的，双端架构一致的动态页面搭建方案。

目录结构如下，业务开发中触及的核心业务代码包括，交互层的(Template/Container/Component)，以及业务微服务(Service)。

### 项目结构

```
Sources/
├── Core/                # 核心基础设施层
│   ├── Base/            # 基础组件和协议
│   ├── Engine/          # 框架引擎核心
│   ├── Data/            # 核心数据模型
│   └── Utils/           # 通用工具类
├── Biz/                 # 业务逻辑层
│   ├── Services/        # 业务服务
│   ├── Components/      # 业务组件
│   ├── Containers/      # 容器组件
│   └── Templates/       # 模板实现
```

### rules

核心 `project rules` 几乎与业务架构一一对应。

```
# TurboDressingFramework Cursor Rules
本目录包含了为TurboDressingFramework项目定制的Cursor Rules，旨在帮助开发者更好
## 规则文件概述
### 1. turbo-framework-architecture.mdc
**适用范围**: 全局应用
**内容**: 框架架构设计、项目结构、核心概念和开发约束
- 三层UI架构原则
- 模块化设计理念
- 服务层架构规范
- 命名规范和开发约束

### 2. component-development.mdc
**适用范围**: Component相关文件 (`**/Component/**/*.swift`, `**/Component
**内容**: 组件开发规则和约束
- 组件层次结构和继承关系
- 生命周期管理规范
- 自定义组件开发流程
- 组件类型规范和最佳实践

### 3.template-development.mdc
**适用范围**: Template相关文件 (`**/Template/**/*.swift`, `**/Template/**
```

**内容**: 模板系统开发规则
- 模板系统架构和协议
- 模板类型体系
- 容器系统管理
- 自定义模板开发规范

### 4. service-development.mdc
**适用范围**: Service相关文件 (`**/Service/**/*.swift`, `**/Service/**/*.
**内容**: 服务开发规则和约束
- 服务架构设计
- 服务创建和注册规范
- 依赖管理最佳实践
- 核心服务类型说明

### 5. pagemodel-configuration.mdc
**适用范围**: Model相关文件 (`**/Model/**/*.swift`, `**/Model/**/*.h`, `*
**内容**: PageModel配置规则
- PageModel结构约束
- 配置规范和数据绑定
- 模板类型配置示例
- 组件配置规范

### 6. ui-tree-structure.mdc
**适用范围**: Template、ViewController、Component相关文件
**内容**: UI树结构和布局管理
- V1和V2版本UI结构
- 容器管理系统
- 子模板系统
- 布局系统和最佳实践

## 使用方法
### 自动应用规则
- `turbo-framework-architecture.mdc` 会在所有请求中自动应用
- 其他规则会根据文件类型自动匹配应用
### 手动应用规则
当需要特定规则的指导时，可以在对话中明确提及：
- "请按照组件开发规则创建新的Component"
- "根据模板开发规则设计新的Template"
- "按照服务开发规则实现新的Service"
### 规则优先级
1. 全局架构规则 (`turbo-framework-architecture.mdc`)
2. 文件类型特定规则（根据文件路径自动匹配）
3. 具体功能规则（根据开发任务手动应用）
## 规则更新
这些规则基于项目的AI知识库生成，包含了：
- 架构描述文档

```
- 编码指南文档
- 实际代码结构分析
当项目架构或开发规范发生变化时，应及时更新相应的规则文件。

## 注意事项
1. **规则冲突**：如果不同规则之间存在冲突，以更具体的规则为准
2. **最佳实践**：规则中包含了大量最佳实践建议，建议仔细阅读
3. **代码示例**：规则中提供了大量代码示例，可以直接参考使用
4. **约束检查**：开发过程中应定期检查是否遵循了相关规则

## 支持的功能
这些规则支持以下开发场景：
- 创建新的Component、Template、Service
- 配置PageModel和Template
- 设计UI树结构和布局
- 实现数据绑定和交互逻辑
- 优化性能和扩展性

通过遵循这些规则，可以确保代码质量、一致性和可维护性。
```

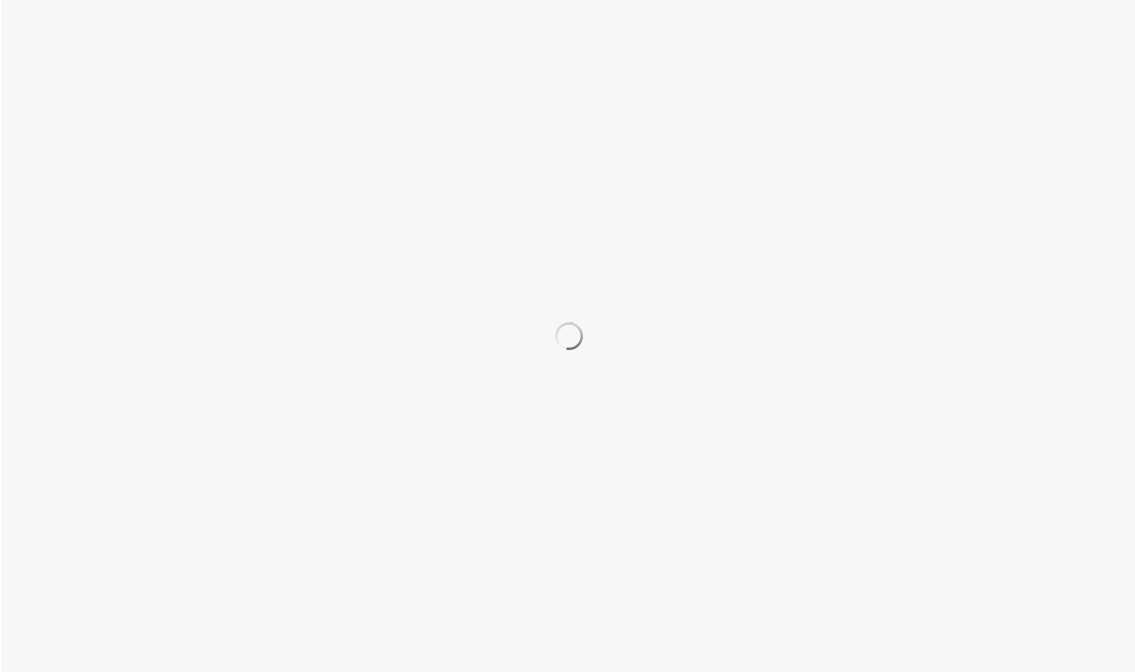
**开发范式**

技术方案双端对焦后，对于可AI Coding的需求，直接共享需求prompt，一键生成(针对部分类命名可做平台测调整)。

这样的开发方式，除了提升了Coding的效率，同时也能一定程度上降低客户端开发时，双端的不一致问题(逻辑差异，消息名不一致，技术方案差异等等)，减少了方案不一致导致后需问题的可能性。

需补充说明，目前这种开发方式更适用于一些新功能的开发场景，且只在双方沟通确认技术方案后，Coding编码阶段由AI介入，减少的是Coding的时间。对于日常的交互bugfix，以及跨端跨模块的交互开发，仍以手动Coding为主（这部分问题的工作往往在频繁的交互调试和方案沟通中，Coding 工作中在需求占比不高）。

以过去三天的工作为例：



**拓展阅读**

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法


大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 4259